How to run the tools as a beginner

Cerberus Framework · local learning guide · 28 October 2026 (Cairo)

> **Start here:** these commands run a disposable lab on your own computer. They do not scan a website or machine you specify. The lab is deliberately vulnerable and listens only on `127.0.0.1` until the command finishes.

What is available?

Integration	In this guide	What a successful run demonstrates
Nmap	Real installed Nmap	Finds the lab's temporary open TCP port.
sqlmap	Real installed sqlmap	Detects the lab's intentional SQL injection.
Metasploit	**Simulated**, not the real program	The adapter exchanges and parses a fabricated XML-RPC response. The simulator does not scan.

The real Metasploit RPC protocol and the current adapter do not match. Installing Metasploit does **not** make the Cerberus integration work with real `msfrpcd`. The app's API, AI agent, and interface do **not** yet start these modules; that wiring is a later phase.

Choose what you need

- For **Nmap alone**, install the backend Python dependencies and Nmap.
- For **sqlmap alone**, install the backend Python dependencies and sqlmap.
- For **Metasploit simulation alone**, only the backend Python dependencies are needed. There is no 1 GB Metasploit download.
- For **all** or **parallel**, both real Nmap and real sqlmap are required; Metasploit is still simulated.

You do not need to start the web app, PostgreSQL, Redis, or a real Metasploit daemon for this lab. The lab prints results but does not save them to the app database.

1. One-time setup: MacBook / zsh

A. Open the correct folder

In VS Code choose **File → Open Folder**, select the project folder containing `backend`, `docs`, and `scripts`, then choose **Terminal → New Terminal**. All commands in this guide start from that project root. Do not type the `$` prompt itself.

B. Prepare the project's Python environment

The pinned project dependencies are intended for **Python 3.11 or 3.12**; 3.12 is recommended. If needed, install it with Homebrew: `brew install python@3.12`. Install Homebrew first from brew.sh if `brew` is not available.

If `backend/.venv` already exists, **do not recreate or delete it**; skip the first line. Install the project packages into that environment:

```
python3.12 -m venv backend/.venv
./backend/.venv/bin/python -m pip install -r backend/requirements.txt
```

C. Install and check the real tools

```
brew install nmap sqlmap
nmap --version
sqlmap --version
```

Install just the tool you want if you are not running both. If a command is not found after installation, reopen the VS Code terminal and follow the PATH troubleshooting section.

D. Your settings and secret

The lab enables its selected integrations **in memory for that run only**; no `.env` file is edited. If no `SECRET_KEY` is configured, the lab generates a temporary random process-only key. If a key is already set but too short, fix the configuration instead. For the full app, configure a permanent private random `SECRET_KEY` of at least 32 characters in `backend/.env`; do not commit it or reuse the example placeholder.

2. One-time setup: Windows PowerShell

Open the project folder in VS Code, then open a **PowerShell** terminal at the project root. Use a Python 3.11/3.12 installation; the example uses 3.12. If `backend\.venv` already exists, skip the first line and do not delete it.

```
py -3.12 -m venv backend\.venv
.\backend\.venv\Scripts\python.exe -m pip install -r backend\requirements.txt
```

Install Nmap

Download and run the official Windows installer from nmap.org/download. Reopen VS Code or the terminal so its PATH refreshes, then check:

```
nmap --version
```

If it is not found, locate the installed `nmap.exe` and set `NMAP_PATH` to its **actual** full path for this terminal; see troubleshooting.

Install sqlmap

Install Git for Windows if `git --version` fails. The following downloads the official sqlmap repository into a new folder in your home directory. If that folder already exists with `sqlmap.py`, skip the clone; otherwise choose a different empty path.

```
git clone --depth 1 https://github.com/sqlmapproject/sqlmap.git "$HOME\cerberus-sqlmap"
$env:SQLMAP_PATH = "$HOME\cerberus-sqlmap\sqlmap.py"
.\backend\.venv\Scripts\python.exe "$env:SQLMAP_PATH" --version
```

`SQLMAP_PATH` is set only for the current PowerShell session; repeat the second line in a new terminal. For the simulated Metasploit run, neither Nmap nor sqlmap needs to be installed.

3. Run one integration at a time

First finish the setup for your platform. Run each command from the project root; copy the line for your operating system. Each run starts and stops its own local lab.

Nmap — real executable

Mac / zsh

```
./backend/.venv/bin/python scripts/live_tests/beginner_tools_lab.py nmap
```

Windows / PowerShell

```
.\backend\.venv\Scripts\python.exe scripts\live_tests\beginner_tools_lab.py nmap
```

Expect `PASS: real Nmap found the local lab's open port` followed by a changing port number.

sqlmap — real executable

Mac / zsh

```
./backend/.venv/bin/python scripts/live_tests/beginner_tools_lab.py sqlmap
```

Windows / PowerShell

```
.\backend\.venv\Scripts\python.exe scripts\live_tests\beginner_tools_lab.py sqlmap
```

Expect `PASS: real sqlmap detected` the intentionally vulnerable `id` parameter. This is a positive result for the **toy lab**, not evidence about your own application.

Metasploit — simulated response only

Mac / zsh

```
./backend/.venv/bin/python scripts/live_tests/beginner_tools_lab.py metasploit
```

Windows / PowerShell

```
.\backend\.venv\Scripts\python.exe scripts\live_tests\beginner_tools_lab.py metasploit
```

Expect `SIMULATED` and a `PASS` for parsing the fake response. No real `msfconsole`, `msfrpcd`, or Metasploit scan was tested.

4. Run them together

Option A — one sequential plan: `all`

This runs **Nmap → sqlmap → simulated Metasploit**, one after the other, in one framework engagement. It does not run the tools at the same instant. Nmap's discovered ports are not automatically passed to the Metasploit module.

Mac / zsh

```
./backend/.venv/bin/python scripts/live_tests/beginner_tools_lab.py all
```

Windows / PowerShell

```
.\backend\.venv\Scripts\python.exe scripts\live_tests\beginner_tools_lab.py all
```

Option B — concurrent, separate engagements: `parallel`

These three independent runs overlap in time against the **same local toy server**. They use separate contexts; results are not shared. This is a concurrency demonstration, not the normal scheduler's behavior.

Mac / zsh

```
./backend/.venv/bin/python scripts/live_tests/beginner_tools_lab.py parallel
```

Windows / PowerShell

```
.\backend\.venv\Scripts\python.exe scripts\live_tests\beginner_tools_lab.py parallel
```

Expected output

A successful `all` or `parallel` run shows a local address, three `PASS` lines, an explicit `SIMULATED` label for Metasploit, and `DONE`. The port changes each time. A `FAILED` message, missing dependency, skipped result, or `ERROR` finding is **not** a successful check. Let the command finish; real sqlmap can take a while.

On normal completion, the local servers stop and sqlmap's temporary output is cleaned up. The report is not saved to PostgreSQL or displayed in the app. Pressing Ctrl+C requests cleanup; a forced termination may leave temporary files.

5. Troubleshooting

If you see…	Try this
`No such file` for the Python path	Open the project root in VS Code; confirm `backend/.venv` exists and use the command for your OS.
`Missing Python package`	Run the setup's pip install with the project's `.venv` Python, not a different Python.
`Missing required tool(s)`	Install the named real tool. `all`/`parallel` need both; simulated Metasploit needs neither.
`nmap` or `sqlmap` not found	Reopen the terminal, verify the binary and PATH, or set the variables below. An inherited environment variable overrides `backend/.env`.
`SECRET_KEY` too short	Fix the existing exported value or private `backend/.env`; use at least 32 random characters. Do not share it.
sqlmap failed or timed out	Verify its version and path, check local security software, and retry. A failed or inconclusive scan is not a clean bill of health.
`msfconsole: command not found`	It is **not needed** for this lab. The simulated run does not verify a real Metasploit installation.
`--target` is rejected	This runner deliberately accepts no custom target; it only uses its disposable loopback lab.

If the binaries are not on PATH

Mac / zsh: after installing with Homebrew, these set paths for this terminal:

```
export NMAP_PATH="$(command -v nmap)"
export SQLMAP_PATH="$(command -v sqlmap)"
```

Windows / PowerShell: replace the first example with the actual `nmap.exe` location; the second uses the sqlmap clone from setup:

```
$env:NMAP_PATH = 'C:\Program Files (x86)\Nmap\nmap.exe'
$env:SQLMAP_PATH = "$HOME\cerberus-sqlmap\sqlmap.py"
```

Check `& $env:NMAP_PATH --version` and `Test-Path $env:SQLMAP_PATH`. These variables apply to the current terminal only. Do not assume the example Nmap location matches your installation.

6. Important files and folders

All paths are relative to the project root. `.env` is private; `.env.example` is a template, not a secret store.

Path	Why it matters
`scripts/live_tests/beginner_tools_lab.py`	Beginner-friendly modes: `nmap`, `sqlmap`, simulated `metasploit`, `all`, and `parallel`. No custom target.
`scripts/live_tests/sqlmap_framework_smoke.py`	More extensive local lab and fake RPC service reused by the beginner runner.
`backend/requirements.txt` and `backend/.venv/`	Backend dependencies and the local Python environment. The virtual environment is not committed.
`backend/.env` / `backend/.env.example`	Private settings / example settings. Exported variables take precedence. Do not commit secrets.
`backend/app/ai_engine/pentest/tools/`	External adapters: `nmap_adapter.py`, `sqlmap_adapter.py`, `metasploit_adapter.py`, and the tool registry.
`backend/app/ai_engine/pentest/modules/tools/`	Framework modules: `nmap_scanner.py`, `sqlmap_scanner.py`, and `metasploit_scanner.py`.
`backend/app/ai_engine/pentest/core/`	Context, scope, safety checks, findings, and evidence contracts.
`backend/app/ai_engine/pentest/engine/`	Module discovery, execution engine, and **sequential** scheduler.
`backend/app/ai_engine/pentest/integration/`	Explicit helpers for converting engagements and persisting findings; not automatically called by the UI.
`backend/tests/unit/`	Framework/tool tests, including `test_sqlmap_integration.py` and `test_beginner_tools_lab.py`.
`docs/SQLMAP_INTEGRATION.md`	More detail on the sqlmap adapter, scope restrictions, and verification.
`docs/HOW_TO_RUN_THE_TOOLS_AS_BEGINNER.md`	Editable source of this guide. The accompanying PDF is the printable copy.
`scripts/render_beginner_tools_guide.py`	Rebuilds the guide PDF from the Markdown source.

The framework also contains built-in web, network, and credential-check modules under `pentest/modules/`. They are **separate from the three external integrations** and are not included in this guide's `all` mode.

7. Boundaries, safety, and upkeep

What the lab does not prove

- The application UI, API, and AI agent are not wired to invoke this framework. Enabling a setting alone does not add a button or start a scan.
- The fake Metasploit service returns test data; it neither runs nor validates a real Metasploit module. The current Cerberus adapter uses XML-RPC, while real `msfrpcd` uses MessagePack.
- Existing framework-wide safety gaps include loose scope/default-port matching, Metasploit option overrides, Nmap wildcard script filtering, and testing windows not enforced for other modules. Do **not** use this lab runner as a production scanner or treat the framework as production-ready.
- sqlmap refuses an explicitly configured testing window instead of enforcing it. It also refuses hostname targets with IP/CIDR exclusions and does not pin DNS for other hostname scans. Approved hours must be enforced externally.
- Only scan systems with explicit written authorization, approved targets, ports, impact limits, and timing. The commands here never accept a real target.

Keeping this guide current

When an external tool is added, removed, or its behavior changes, the contributor must update the tool table, prerequisites, individual/combined examples, limitations, important files, and tests; add a safe local fixture if possible. If it cannot be verified, say so. Regenerate and visually inspect the PDF in the **same change**. A test checks registered-tool markers and the source hash in the PDF, but it cannot prove the instructions are correct.

Maintainer: rebuild the printable PDF

Install developer requirements and WeasyPrint's native OS dependencies if needed. From the project root:

Mac / zsh

```
./backend/.venv/bin/python -m pip install -r backend/requirements-dev.txt
./backend/.venv/bin/python scripts/render_beginner_tools_guide.py
```

Windows / PowerShell

```
.\backend\.venv\Scripts\python.exe -m pip install -r backend\requirements-dev.txt
.\backend\.venv\Scripts\python.exe scripts\render_beginner_tools_guide.py
```

The canonical output is `docs/how to run the tools as beginner.pdf`. Inspect each page and verify the commands on the relevant operating system before publishing.

8. Verification and useful links

What was checked for this edition

The real Nmap and sqlmap integrations were exercised individually and together on a disposable localhost lab. The sequential plan and separate concurrent engagements were checked, with a **simulated** Metasploit endpoint. Backend tests and a frontend build were also run in a Linux sandbox. The Mac and Windows instructions were written for those systems but were **not executed on macOS or Windows** for this edition.

The lab runner does not need the full app, and a successful frontend build does not prove that the tools are connected to the interface. The demo checks recognizable results, not merely a process exit code or an empty error list.

Optional deeper check

The existing broader sqlmap lab tests GET, POST, redirects, parameter handling, and concurrent engagements. It needs a configured backend secret and real sqlmap; it also uses real Nmap if available and **fake** Metasploit. It does not persist results.

Mac / zsh

```
./backend/.venv/bin/python scripts/live_tests/sqlmap_framework_smoke.py
```

Windows / PowerShell

```
.\backend\.venv\Scripts\python.exe scripts\live_tests\sqlmap_framework_smoke.py
```

The existing smoke script reads `backend/.env` or environment variables; unlike the beginner runner, it does not generate a temporary secret. If Nmap is absent, that part is skipped. Confirm the output rather than assuming every integration ran.

Official references

- Nmap downloads and Windows installer · Homebrew Nmap
- Official sqlmap repository · Homebrew sqlmap
- Rapid7 Metasploit RPC documentation
- For separate, optional Metasploit installation, older instructions may suggest `brew install --cask metasploit`. Homebrew currently marks that cask disabled, so it may fail. It is not needed for the lab and would not fix the protocol mismatch.

Remember: this guide is a local learning path. Do not confuse a successful simulated integration test with a real authorized security assessment.